

Laborator 6

Procese

6.1 Scopul lucrării

Laboratorul curent prezintă conceptul de proces în Linux, împreună cu cele mai folosite apeluri de sistem pentru manipularea proceselor.

6.2 Fundamente teoretice

6.2.1 Conceptul de proces

Un proces este o entitate ce reprezintă un program care rulează. Pe lângă codul programului, procesul mai conține și date, descriptori de fișiere deschise, fire de execuție, precum și orice alte resurse care țin de execuția programului.

Fiecare proces are un identificator unic, numit PID (eng. *process identifier*), care poate fi folosit pentru a interacționa cu procesul.

Cu excepția procesului cu PID-ul 0, fiecare proces are un părinte (procesul care l-a creat). Relația tată-fiu generează în mod implicit o structură de arbore pentru procesele din sistem.

În cazul în care un proces (părinte) își încheie execuția în timp ce unii din copiii săi încă rulează, spunem că aceștia devin *orfani*. Copii orfani sunt “adoptați” în mod automat de procesul cu PID-ul 1.

În cazul în care un proces (copil) își încheie execuția în timp ce părintele lui încă se execută (și nu așteaptă după terminarea copilului respectiv), procesul va fi pus în starea “*zombie*”, adică proces terminat, dar pentru care se păstrează încă informații. Acele informații sunt șterse și, implicit, procesul zombie șters, când părintele se termină sau cere sistemului de operare informații despre copilul terminat.

6.2.2 Crearea de procese și obținerea identificatorilor

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);

pid_t getpid(void);
pid_t getppid(void);
```

6.2.2.1 fork

În Linux, `fork` este singurul apel de sistem care dă naștere unui nou proces. La apelul acestuia se va crea un proces copil, identic cu părintele (atât din punctul de vedere al codului, cât și al resurselor), care va continua execuția imediat după apelul de sistem. Singura diferență este valoarea returnată de `fork()`. Astfel, în procesul copil se va returna 0, pe când în procesul părinte se va returna PID-ul procesului copil. În caz de eroare (nu se poate crea procesul fiu), se returnează -1.

6.2.2.2 getpid și getppid

Apelurile de sistem `getpid` și `getppid` sunt utilizate pentru a obține PID-ul propriu, respectiv PID-ul procesului părinte. Tipul returnat de ambele apeluri de sistem este `pid_t`.

6.2.2.3 Exemplu

Dorim să scriem un program care dă naștere unui proces fiu. Procesul părinte își va afișa PID-ul propriu și pe cel al copilului, în timp ce procesul fiu își va afișa PID-ul propriu și pe cel al părintelui.

```

1  int main(void)
2  {
3      pid_t pid;
4
5      pid = fork();
6      if(pid == -1) {
7          perror("Could not create child process");
8          return -1;
9      } else if(pid == 0) {
10         //this code is executed only by child
11         printf("[CHILD] My PID is %d. My parent's PID is %d.\n",
12             getpid(), getppid());
13     } else {
14         //this code is executed only by parent
15         printf("[PARENT] My PID is %d. I created a child with PID %d.\n",
16             getpid(), pid);
17         wait(NULL);
18     }
19     return 0;
20 }
```

Programul începe prin a declara variabila `pid`, de tip `pid_t` (linia 3). Acest tip, în practică, este un întreg cu semn (`int`). La linia 5 se efectuează apelul de sistem, al cărui rezultat se salvează în variabila `pid`. De aici încolo, se execută în paralel atât procesul părinte, cât și procesul copil.

Se verifică întâi dacă apelul de sistem a reușit. În caz contrar, valoarea returnată este -1 (linia 6) și se iese cu un mesaj de eroare (linia 7).

În cazul în care apelul de sistem a reușit, valoarea returnată este fie 0, caz în care ne aflăm în procesul copil, fie un număr mai mare decât 0, caz în care ne aflăm în procesul părinte.

În cazul în care ne aflăm în procesul copil, se afișează PID-ul propriu și cel al părintelui, valori obținute apelând `getpid()` și `getppid()` (liniile 11-12).

În cazul procesului părinte (liniile 14-17), PID-ul propriu se obține tot apelând `getpid()`, în timp ce PID-ul copilului a fost returnat de `fork()`, deci se găsește în variabila `pid`. Procesul părinte va aștepta și terminarea execuției copilului, apelând `wait()` (linia 17), a cărui funcționalitate va fi descrisă în secțiunea următoare.

La execuția programului se va afișa un text similar cu cel de mai jos.

```

[PARENT] My PID is 14982. I created a child with PID 14983.
[CHILD] My PID is 14983. My parent's PID is 14982.
```

Valorile PID-urilor pot fi diferite de la un sistem la altul și de la o rulare la alta. De asemenea, ordinea în care se afișează cele două linii poate să difere. În schimb, valorile de pe prima linie trebuie să fie aceleași cu valorile de pe a doua linie, inversate.

6.2.3 Terminarea proceselor și așteptarea terminării copiilor

```

#include <sys/types.h>
#include <sys/wait.h>
```

```
#include <unistd.h>
#include <stdlib.h>

void exit(int status);

pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

6.2.3.1 exit

Pentru a închide procesul curent se poate folosi apelul de sistem **exit**. Acesta primește ca parametru un număr care indică starea programului la ieșire. Valorile valide pentru parametrul **status** sunt numere de la 0 la 255. Prin convenție, starea 0 indică faptul că execuția s-a încheiat cu succes, în timp ce alte valori indică erori.

6.2.3.2 wait și waitpid

În mod normal, un proces părinte creează copii pentru a le delega unele sarcini și de obicei așteaptă terminarea execuției acestora.

Apelul de sistem **waitpid** primește ca parametru PID-ului procesului copil pentru care se așteaptă terminarea. Opțional, se primește și un pointer la o variabilă de tip **int**, prin parametrul **wstatus** în care se va salva starea cu care procesul copil s-a terminat (dacă nu avem nevoie de stare, transmitem NULL). Parametrul **options** se folosește pentru a indica niște condiții în care să nu se aștepte terminarea copilului (pentru mai multe detalii consultați pagina de manual) și are valoarea 0 dacă dorim să așteptăm tot timpul.

Apelul de sistem **wait** are un comportament similar cu **waitpid**, dar nu se specifică un copil anume, ci se va aștepta terminarea oricărui copil.

Pentru a folosi starea cu care s-a terminat procesul copil, nu vom folosi direct valoarea completată în parametrul **wstatus**, ci o vom extrage din acesta utilizând macro-ul **WEXITSTATUS**.

6.2.3.3 Exemplu

Un proces părinte va declara două variabile **x** și **y** pe care le va inițializa. Același proces va da naștere la 3 procese copii, fiecare din acestea efectuând una dintre operațiile *adunare*, *scădere*, respectiv *înmulțire* între cele două variabile și va transmite rezultatul părintelui folosind starea sau codul de ieșire (en. *exit status*). Înainte să facă calculul, fiecare proces copil va aștepta un timp aleator, cel mult o jumătate de secundă.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <sys/wait.h>
5 #include <unistd.h>
6
7 int main(void)
8 {
9     pid_t pid1=-1, pid2=-1, pid3=-1;
10    int x=5, y=3, z=0;
11    int status1=0, status2=0;
12
13    pid1 = fork();
14    if(pid1 == 0) {
15        //1st child
16        srand(getpid());
17        usleep(random() % 500000);
18        z = x + y;
19        exit(z);
20    }
```

```

21     pid2 = fork();
22     if(pid2 == 0) {
23         //2nd child
24         srand(getpid());
25         usleep(random() % 500000);
26         z = x - y;
27         exit(z);
28     }
29     pid3 = fork();
30     if(pid3 == 0) {
31         //3rd child
32         srand(getpid());
33         usleep(random() % 500000);
34         z = x * y;
35         exit(z);
36     }
37     waitpid(pid2, &status1, 0);
38     printf("The result of the subtraction is %d.\n", WEXITSTATUS(status1));
39     wait(&status1);
40     wait(&status2);
41     printf("The result of the other two operations are %d and %d.\n",
42           WEXITSTATUS(status1), WEXITSTATUS(status2));
43
44     return 0;
45 }

```

Părintele începe prin inițializarea variabilelor `x` și `y` (linia 10), variabile ce vor fi moștenite și de copii. Pentru a crea fiecare proces copil se apelează `fork()` (liniile 13, 21 și 29).

Fiecare din cei 3 copii trebuie să aștepte un timp aleator. Pentru a genera numere aleatoare, de obicei se inițializează generatorul de numere aleatoare folosind funcția `srand()`, o singură dată, la începutul programului. Apoi, de fiecare dată când e nevoie de un număr aleator se apelează funcția `rand()`. Numerele aleatoare generate nu sunt cu adevărat aleatoare, ci sunt determinate de starea generatorului. De obicei se apelează `srand(time(NULL))` la începutul programului, astfel la fiecare rulare a programului generatorul va fi inițializat cu altă valoare (în funcție de timpul curent). Dacă am folosi această abordare, cei 3 copii ar moșteni aceeași stare a generatorului și ar genera același timp de așteptare. Prin urmare, atunci când lucrăm cu numere aleatoare în procesele copii, ar trebui ca fiecare proces să își inițializeze generatorul de numere aleatoare. Deasemenea, nu putem folosi `time(NULL)` (timpul curent, în secunde) ca valoare de inițializare, deoarece sunt șanse ca toate cele trei procese copil să apeleze în aceeași secundă și să inițializeze cu aceeași valoare. Ar trebui să inițializăm cu ceva unic pentru fiecare proces, cum ar fi PID-ul acestuia.

După ce fiecare proces copil își execută operația, va transmite rezultatul la părinte, folosind apelul de sistem `exit()` (liniile 19, 27 și 35).

Procesul părinte așteaptă întâi după al doilea copil, apelând `waitpid()` la linia 37. Aici, procesul părinte este blocat până ce al doilea copil își termină execuția. Chiar dacă un alt copil își termină mai repede execuția, părintele nu continuă până nu primește rezultatul de la al doilea copil (cel care a efectuat scăderea).

Pentru copiii 1 și 3 se așteaptă folosind apelul de sistem `wait` (liniile 39 și 40). Deoarece nu se știe care din cei doi copii își va încheia primul execuția (sau și-a încheiat-o deja, înainte de copilul 2), rezultatele pot să vină în orice ordine, deci `status1` poate fi rezultatul adunării iar `status2` rezultatul înmulțirii, sau invers.

6.2.4 Lansarea în execuție a programelor

```

#include <unistd.h>

int execl(const char *path, const char *arg, ... /* (char *) NULL */);

```

```
int execlp(const char *file, const char *arg, ... /* (char *) NULL */);
int execlv(const char *path, char *const argv[]);
int execlvp(const char *file, char *const argv[]);
```

6.2.4.1 Familia de funcții exec

Familia de funcții **exec** are rolul de a **înlocui** imaginea procesului curent cu un alt program (fișier executabil), a cărui cale este transmisă ca argument. După execuția noului program, nu se revine în codul programului apelant. Deoarece nu se creează un proces nou, ci se înlocuiește cel curent, anumite resurse, cum ar fi descriptorii de fișiere deschise, se moștenesc.

Funcția **execl()** primește un număr variabil de argumente, primul fiind calea spre un executabil, iar celelalte sunt argumentele pe care executabilul le primește în linia de comandă, inclusiv numele acestuia (ca argument 0). Ultimul argument trebuie să fie **NULL**, deoarece funcțiile variadice în C (funcții cu număr variabil de argumente) nu știu câte argumente au primit și trebuie să își poată da seama unde să se oprească.

De exemplu, dacă vrem să scriem un program care afișează permisiunile fișierului primit ca parametru, putem executa programul **ls**, ca în exemplul de mai jos:

```
1 | ...
2 | execl("/bin/ls", "ls", "-l", argv[1], NULL);
3 | perror("execl failed");
4 | ...
```

Primul argument al funcției **execl** este calea spre executabilul **ls** ("**/bin/ls**"). Urmează numele comenzii ("**ls**"), argumentele acesteia ("**-l**" și **argv[1]**), respectiv santinela **NULL**. Spre deosebire de alte apeluri de sistem, nu e nevoie să verificăm valoarea returnată de **execl**. În caz de succes, programul primit ca parametru se execută fără să se revină din acesta. În caz de eroare, se execută pur și simplu linia următoare din program, deci dacă execuția a ajuns în acel punct, știm sigur că am avut de-a face cu o eroare.

Dacă în loc să specificăm calea completă spre executabilul **ls** i-am fi scris doar numele, funcția **execl** nu l-ar fi găsit. Pentru un comportament similar cu cel al interpretorului de comenzi (executabilul să fie căutat în variabila de mediu **PATH**), vom folosi funcția **execlp()**, care primește aceleași argumente ca și **execl()**. În acest caz, exemplul de mai sus ar deveni:

```
1 | ...
2 | execlp("ls", "ls", "-l", argv[1], NULL);
3 | perror("execlp failed");
4 | ...
```

Atunci când argumentele unui program se construiesc programatic trebuie să transmitem un vector cu acestea, în loc să utilizăm o funcție variadică. Funcțiile **execv** și **execvp** funcționează ca și **execl** și **execlp**, cu diferența că primesc un vector cu argumentele. Ultimul element al vectorului trebuie deasemenea să fie **NULL**. Exemplul de mai jos are același rezultat cu cele anterioare, dar utilizează funcția **execv**.

```
1 | ...
2 | char *arguments[4];
3 | arguments[0] = "ls";
4 | arguments[1] = "-l";
5 | arguments[2] = argv[1];
6 | arguments[3] = NULL;
7 | execv("/bin/ls", arguments);
8 | perror("execv failed");
9 | ...
```

6.2.4.2 Exemplu: redirectare

Dorim să listăm toate elementele aflate în directorul rădăcină **/** (nerecursiv) într-un fișier primit ca argument în linie de comandă, folosind comanda **ls**. Practic vrem să scriem un program C, care realizează operația de redirectare din comanda **ls / > file.txt**.

Ne vom folosi de faptul că programele executate cu familia de funcții **exec** moștenesc fișierele deschise și descriptorii acestora.

```

1 | int main(int argc, char **argv)
2 | {
3 |     pid_t pid;
4 |     int fd, status;
5 |
6 |     if(argc != 2) {
7 |         fprintf(stderr, "Usage: %s <result_file>\n", argv[0]);
8 |         return 1;
9 |     }
10 |
11 |     pid = fork();
12 |     if(pid == -1) {
13 |         perror("Could not create child process");
14 |         return -1;
15 |     } else if(pid == 0) {
16 |         fd = open(argv[1], O_CREAT | O_TRUNC | O_WRONLY, 0644);
17 |         if(fd < 0) {
18 |             perror("Could not open the result file");
19 |             exit(1);
20 |         }
21 |         dup2(fd, 1); //use fd as standard output
22 |         //ls will inherit the standard output
23 |         execlp("ls", "ls", "/", NULL);
24 |         perror("execlp failed");
25 |         exit(2);
26 |     } else {
27 |         wait(&status);
28 |         if(WEXITSTATUS(status) == 0) {
29 |             printf("Command completed successfully!\n");
30 |         } else {
31 |             printf("Command failed!\n");
32 |         }
33 |     }
34 |     return 0;
35 | }
```

La liniile 6–9 se verifică faptul că programul a primit un argument în linia de comandă. La linia 11 se apelează **fork()**, dând naștere unui proces copil. În procesul copil **fork()** a returnat 0, deci codul executat este cel din liniile 16–25. În primul rând, se deschide fișierul primit ca argument în linia de comandă (linia 16). Chiar dacă acest argument a fost primit de părinte, copilul este în acest moment o clonă a părintelui, deci va avea aceleași valori ale argumentelor.

În continuare, vrem să redirectionăm conținutul care s-ar scrie în mod normal pe ecran către fișierul pe care tocmai l-am deschis. Pentru aceasta vom folosi apelul de sistem **dup2()** (linia 21). Acest apel de sistem duplică primul descriptor de fișier primit ca prim parametru (**fd**), utilizând descriptorul de fișier primit ca al doilea parametru. În acest caz, al doilea parametru este 1, deoarece vrem să folosim fișierul deschis pe post de ieșire standard (**stdout**), care are tot timpul descriptorul de fișier 1. Vechiul fișier indicat de descriptorul 1 va fi închis.

La linia 23 apelăm **execlp()** pentru a executa comanda dorită. În cazul în care **execlp()** nu se execută cu succes, afișăm eroarea (linia 24) și închidem procesul copil (linia 25).

Procesul părinte așteaptă terminarea copilului (linia 27) și, în funcție de starea de ieșire a acestuia, afișează dacă comanda s-a executat cu succes sau nu. Mesajele afișate de părinte vor apărea pe ecran, deoarece redirectarea ieșirii standard (adică a descriptorului de fișier 1) s-a făcut doar pentru procesul copil.

6.3 Mersul lucrării

6.3.1 Întrebări recapitulative

1. Ce returnează apelul de sistem `fork`?
2. Cum poate un proces să își afle PID-ul propriu? Dar PID-ul părintelui?
3. Cum se transmite starea de terminare a unui proces copil părintelui acelui copil?
4. Ce se întâmplă dacă un proces copil își încheie execuția înainte ca părintele să apeleze `wait`? Dar invers?
5. De ce execuția liniei imediat următoare de după o funcție din familia `exec` semnifică o eroare?
6. Care este diferența dintre primul și al doilea argument al funcției `execl`?
7. Dacă dorim să executăm comanda `cat` fără să știm exact unde se află programul respectiv în sistemul de fișiere, folosim funcția `execv` sau `execvp`? Justificați.

6.3.2 Probleme rezolvate

1. (*l06p1_parent_child.c*) Programul este descris în secțiunea 6.2.2.3.
2. (*l06p2_wait.c*) Programul este descris în secțiunea 6.2.3.3.
3. (*l06p3_redirect.c*) Programul este descris în secțiunea 6.2.4.2.

6.3.3 Probleme propuse

1. (25p) Implementați ierarhia de procese din Figura 6.1, în care programul principal P_1 dă naștere la două procese copil, P_2 și P_3 , fiecare din acestea dând naștere unui nou proces copil, P_4 , respectiv P_5 . Fiecare proces își va afișa propriul PID, PID-ul părintelui, iar în caz că are copii, va aștepta terminarea acestora.

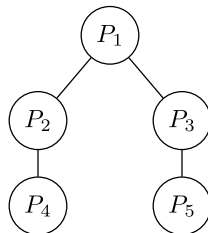


Figura 6.1: Ierarhia de procese

Pentru a vizualiza ierarhia de procese, apăsați în P_4 și P_5 `sleep(60)`. Acest apel de sistem va ține procesele deschise timp de 1 minut. În acest timp, într-un alt terminal, rulați comanda `ps tree -p <PID_P1>`, înlocuind `<PID_P1>` cu PID-ul afișat de procesul P_1 .

2. (25p) Implementați două programe, *client.c* și *server.c*. Programul client afișează un prompt (`>`) și citește de la tastatură doi întregi și un caracter care poate fi '+' sau '-'. După citire se pornește un proces copil, care execută programul *server*, transmițându-i datele citite ca argumente în linia de comandă. Programul *server* va efectua operația primită (adunare sau scădere) pe cele două numere și va transmite rezultatul înapoi părintelui, folosind funcția `exit()` (presupunem că rezultatele sunt numere pozitive între 0 și 255). Părintele va aștepta terminarea copilului, va afișa rezultatul pe ecran și va afișa din nou prompt-ul.

3. (25p) Modificați codul de mai jos pentru a afișa numărul total de procese pornite. Fiecare copil va număra câte procese a creat (împreună cu descendenții săi) și va transmite acest număr părintelui prin exit status. Doar procesul inițial, va afișa pe ecran totalul. Propuneți o soluție care să funcționeze și pentru valori mai mari ale lui `n` (de exemplu 10).

```
1 |   int i, n=7;
2 |   for(i=0; i<n; i++) {
3 |       fork();
4 |   }
```

4. (25p) Scrieți un program care numără câte linii ce conțin secvența "abcd" are fișierul primit ca argument în linia de comandă, apelând `grep` și `wc`. Ar trebui să executați o comandă similară cu `grep abcd $FILENAME | wc -l`. Operatorul `|` (*pipe*) transmite output-ul primei comenzi la input-ul celei de-a doua. Vom folosi un fișier temporar pentru a salva acest output. Puteți utiliza apelul de sistem `dup2()`, ca să redirectați standard output-ul, respectiv standard input-ul unui program la un descriptor de fișier deschis.

Notă: Analogia cu operatorul *pipe* (`|`) nu este perfectă. Acest operator permite execuția în paralel a celor două programe, cel de-al doilea așteptând eventual ca primul să scrie date (se vor prezenta mai multe detalii la laboratorul 11). În cazul nostru, pentru ca numărarea liniilor să funcționeze corect trebuie ca al doilea program (`wc`) să fie rulat doar după ce primul (`grep`) își încheie execuția.